SimplexLab: a pasta `algoritmos`

O que cada arquivo faz, função por função, e em qual etapa do método cada uma se encaixa.

1. Visão geral

A pasta `simplexlab/algoritmos` é o "cérebro" do SimplexLab: é onde os problemas de Programação Linear são resolvidos. A interface (telas, botões, gráfico) fica em outra pasta e só **mostra** o que estes arquivos calculam. Todas as contas são feitas com **frações exatas** (`Fraction` do Python), por isso o programa mostra `1/3` em vez de `0,3333`, igual à resolução no caderno.

Arquivo	O que é, em uma frase
`resolvedor.py`	A "porta de entrada": recebe o problema e o método escolhido e decide qual algoritmo roda.
`algoritmo_pl.py`	Peças comuns a todos os métodos: os resultados possíveis (ótimo, ilimitado, inviável), o formato das etapas e a **montagem da forma padrão**.
`algoritmo_simplex.py`	O **Método Simplex** tabular: o quadro, quem entra, quem sai, pivô e as iterações.
`algoritmo_duas_fases.py`	O **Simplex de Duas Fases**, usado quando há restrições `≥` ou `=`. Reaproveita as iterações do Simplex.
`algoritmo_grafico.py`	O **Método Gráfico** (só para 2 variáveis): retas, região viável, vértices e valor de Z em cada vértice.
`__init__.py`	Arquivo vazio. Só avisa ao Python que a pasta é um pacote (permite `import simplexlab.algoritmos...`).

Como os arquivos se conectam

Interface: o usuário digita o modelo, escolhe "Simplex" ou "Gráfico" e clica em Resolver.

▼

resolvedor.py → `resolver(problema, metodo)` escolhe o algoritmo.

▼

algoritmo_grafico.py	**algoritmo_simplex.py**	**algoritmo_duas_fases.py**
se o método for Gráfico	se for Simplex e só houver $\leqslant$	se for Simplex e houver $\geqslant$ ou $=$

▼

Os três usam **algoritmo_pl.py** e devolvem um `Resultado` com a lista de `Etapa`s, que a interface mostra uma por uma.

2. `resolvedor.py`: quem decide o método

Tem uma única função, `resolver(problema, metodo)`, com esta regra:

1. Se o usuário escolheu **Gráfico**, chama `MetodoGrafico().resolver(problema)`.
2. Se escolheu **Simplex**, monta a forma padrão e olha se apareceu alguma **variável artificial**:
   - apareceu (há restrição $\geqslant$ ou $=$) → usa o **Duas Fases**;
   - não apareceu (todas são $\leqslant$) → usa o **Simplex comum**.

Ou seja, o usuário não precisa saber qual Simplex usar: o programa escolhe sozinho.

3. `algoritmo_pl.py`: as peças comuns e a forma padrão

Situações possíveis do problema

Três constantes dizem como a resolução terminou: `OTIMO` ("Solução ótima encontrada"), `ILIMITADO` ("sem solução ótima finita") e `INVIAVEL` ("não existe solução que atenda todas as restrições"). O dicionário `DESCRICAO_STATUS` guarda a frase que aparece na tela.

Classes que "carregam" a resolução

Classe	Para que serve
`Etapa`	Um passo da resolução que aparece na tela: título, explicação em texto e, se tiver, uma tabela (quadro do Simplex ou tabela do gráfico). Guarda também qual linha e coluna devem ficar **destacadas** (linha pivô e coluna pivô).

Classe	Para que serve
`Resultado`	O resultado final: método usado, situação (ótimo/ilimitado/inviável), valor ótimo de Z, valores de x1, x2…, valores das folgas/excessos, lista de etapas e observações (ex.: "há soluções ótimas alternativas").
`ResultadoGrafico`	Igual ao `Resultado`, mais o que o gráfico precisa: lista de vértices, ponto(s) ótimo(s) e se a região é ilimitada.
`FormaPadrao`	O problema já convertido: nomes das colunas, matriz `[A | b]`, base inicial, custos e quais colunas são artificiais.

`montar_forma_padrao(problema)`: etapa "Forma padrão"

É a **primeira etapa do Simplex** (e do Duas Fases): transformar as desigualdades em igualdades. Ela faz, em ordem:

1. **Min → Max.** Se o problema é de minimização, multiplica a função objetivo por −1 (resolve Max(−Z)). No final o sinal do valor ótimo é trocado de volta.
2. **Lado direito negativo.** Chama `_ajustar_lado_direito`: se algum `b` for negativo, multiplica a restrição inteira por −1 e inverte o sinal (`⩽` vira `⩾`). O Simplex precisa de `b ≥ 0` para começar.
3. **Variáveis auxiliares**, conforme o tipo de cada restrição:

Restrição	O que é somado/subtraído	Quem fica na base inicial
⩽	+ folga `f`	a folga
⩾	− excesso `e` + artificial `a`	a artificial
=	+ artificial `a`	a artificial

4. **Monta as colunas** na ordem: variáveis originais, folgas/excessos e artificiais. Depois monta cada linha `[A | b]`.

Exemplo real que o programa gera para `max z = 3x1 + 5x2` com `x1 ≤ 4`, `2x2 ≤ 12`, `3x1 + 2x2 ≤ 18`:

```
Restrições na forma padrão:
    R1: x1 + f1 = 4
    R2: 2x2 + f2 = 12
    R3: 3x1 + 2x2 + f3 = 18

Variáveis de folga: f1, f2, f3
Base inicial: f1, f2, f3
```

4. `algoritmo_simplex.py`: o Método Simplex

Usado quando todas as restrições são `⩽` (base inicial só de folgas). O arquivo tem duas classes: `QuadroSimplex` (o quadro e as operações sobre ele) e `MetodoSimplex` (o passo a passo do método).

Cada função e o passo da aula a que ela corresponde

Função	Passo do método	O que faz
`MetodoSimplex.resolver`	Montar o quadro I	Chama a forma padrão, monta a linha Z como `Z - 3x1 - 5x2 = 0` (custos com sinal trocado) e cria o `QuadroSimplex`.
`coluna_que_entra`	1º Quem ENTRA	Procura o **coeficiente mais negativo** da linha Z. Se não houver negativo, devolve `None`: o quadro é **ótimo** (critério de parada).
`razoes`	2º Teste da razão	Calcula `b ÷ coeficiente da coluna que entra` em cada linha. Se o coeficiente for 0 ou negativo, a linha **não entra no teste** ("não convém").
`linha_que_sai`	2º Quem SAI (linha pivô)	Escolhe a linha com a **menor razão positiva**. Se nenhuma linha tiver razão válida, devolve `None`: o problema é **ilimitado**.
`_explicar_pivo`	3º Elemento pivô	Escreve o texto da etapa: quem entra, a conta de cada razão, quem sai e o **elemento pivô** (cruzamento da linha que sai com a coluna que entra).
`pivotear`	4º NLP 5º Novas linhas	Divide a linha pivô pelo pivô (**NLP = linha pivô ÷ EP**). Depois, em cada outra linha (inclusive Z), faz `linha - k × NLP`, onde `k` é o número dessa linha na coluna pivô, o que zera a coluna. Por último troca a variável básica daquela linha.
`_subtrair_multiplo`	5º (a conta)	A conta `linha - k × NLP` de uma linha só. É a mesma "soma empilhada" da aula: `NLP × (-k) + linha antiga = nova linha`. Linha com `k = 0` não muda.
`executar_iteracoes`	Repetir os passos	O laço principal: 1º → 2º → registra o quadro → pivoteia → volta ao 1º, até ficar ótimo ou ilimitado (limite de 100 iterações por segurança).
`registrar`	Mostrar o quadro	Tira uma "foto" do quadro atual e vira uma `Etapa` com a linha e a coluna pivô destacadas.
`valores_variaveis`	Leitura VB / VNB	Variável básica vale o `b` da sua linha; não básica vale 0.
`valor_objetivo`	Valor de Z	É o número da coluna `b` na linha Z.
`montar_resultado`	Conclusão	Lê o quadro final: valores de x1, x2…, folgas, Z (troca o sinal de volta se era minimização) e checa soluções múltiplas.
`nao_basicas_com_custo_zero`	Caso especial	No quadro ótimo, variável **não básica com 0 na linha Z** indica **soluções ótimas alternativas** (múltiplas).

Função	Passo do método	O que faz

`_explicar_otimo`
`_explicar_ilimitado`
`_explicar_solucao`
	Textos	Montam as explicações que aparecem na tela em cada caso.

`ajustar_linha_objetivo`
`remover_colunas`
`remover_linha`
	Usadas no Duas Fases	Zerar na linha objetivo os coeficientes das variáveis básicas, tirar as colunas artificiais e tirar restrições redundantes.

Diferença de apresentação em relação ao quadro da aula

No programa a **linha Z fica embaixo** (última linha) e cada linha mostra à esquerda qual é a variável básica (coluna "Base"). No quadro do professor a linha Z é a **1ª linha** e as básicas são lidas pelas colunas de identidade (um 1 e o resto 0). Os números e as contas são os mesmos; muda só a posição.

Proteção contra ciclagem (regra de Bland)

Em problemas degenerados o Simplex pode voltar a um quadro que já visitou e ficar girando para sempre. `executar_iteracoes` guarda as bases já vistas; se uma se repetir, passa a usar a **regra de Bland**: entra a **primeira** variável com coeficiente negativo e, no empate da razão, sai a de menor índice. Isso garante que o método termina.

Exemplo: o que o programa mostra

Mesmo problema da seção anterior (`max z = 3x1 + 5x2`). Em amarelo, a linha e a coluna pivô; com contorno, o elemento pivô.

Base	x1	x2	f1	f2	f3	b
f1	1	0	1	0	0	4
f2	0	**2**	0	1	0	12
f3	3	2	0	0	1	18
Z	**-3**	**-5**	**0**	**0**	**0**	**0**

Quadro inicial: entra x2 (−5 é o mais negativo); razões 12/2 = 6 e 18/2 = 9, então sai f2; pivô = 2.

```
Variável que entra: x2 (coeficiente mais negativo na linha Z: -5)

Teste da razão (b / coluna x2):
    f1: 4 / 0  →  não entra no teste
    f2: 12 / 2 = 6   ← menor
    f3: 18 / 2 = 9

Variável que sai: f2
Elemento pivô: 2
```

Base	x1	x2	f1	f2	f3	b
f1	1	0	1	0	0	4
x2	0	1	0	1/2	0	6
f3	**3**	0	0	-1	1	6
Z	**-3**	**0**	**0**	**5/2**	**0**	**30**

Iteração 1: NLP = linha de f2 ÷ 2; as outras linhas foram zeradas na coluna x2. Agora entra x1 e sai f3 (pivô 3).

Base	x1	x2	f1	f2	f3	b
f1	0	0	1	1/3	-1/3	2
x2	0	1	0	1/2	0	6
x1	1	0	0	-1/3	1/3	2
Z	**0**	**0**	**0**	**3/2**	**1**	**36**

Iteração 2: nenhum negativo na linha Z → quadro ótimo. x1 = 2, x2 = 6, Z = 36.

5. `algoritmo_duas_fases.py`: Simplex de Duas Fases

Quando há restrições `⩾` ou `=`, a forma padrão coloca **variáveis artificiais** na base. Elas não existem no problema real, então precisam ser zeradas antes de resolver de verdade. A classe `MetodoDuasFases` **herda** de `MetodoSimplex`: as iterações (quem entra, quem sai, pivô) são exatamente as mesmas da seção 4.

Função	Etapa do método	O que faz
`resolver`	Roteiro geral	Forma padrão → Fase 1 → se for inviável, para aqui → prepara a Fase 2 → roda o Simplex → monta o resultado.
`_executar_fase_1`	**Fase 1**	Cria a linha **W** (minimizar `W = a1 + a2 + …`), zera nela as artificiais que estão na base (`ajustar_linha_objetivo`) e roda as iterações do Simplex. No final: se **W > 0**, não deu para zerar as artificiais → **problema inviável**; se **W = 0**, achou uma solução viável e segue para a Fase 2.
`_retirar_artificiais_da_base`	Entre as fases	Se alguma artificial terminou a Fase 1 na base valendo 0, troca por outra variável da mesma linha. Se a linha for toda zero, a restrição era **redundante** e é removida.
`_preparar_fase_2`	**Fase 2** (montagem)	Remove as colunas artificiais, coloca a **função objetivo original** na linha Z e zera os coeficientes das variáveis básicas nessa linha.
`executar_iteracoes` (herdada)	**Fase 2** (iterações)	Resolve normalmente, partindo da base que a Fase 1 encontrou.

Exemplo: `min z = 4x1 + x2`, com `3x1 + x2 = 3`, `4x1 + 3x2 ≥ 6`, `x1 + 2x2 ≤ 4`

```
Problema de minimização: resolvemos Max (-Z), multiplicando a função objetivo por -1. No
final o sinal do valor ótimo é trocado de volta.

Restrições na forma padrão:
    R1: 3x1 + x2 + a1 = 3
    R2: 4x1 + 3x2 - e2 + a2 = 6
    R3: x1 + 2x2 + f3 = 4

Variáveis de folga: f3
Variáveis de excesso: e2
Variáveis artificiais: a1, a2
Base inicial: a1, a2, f3
```

Base	x1	x2	e2	f3	a1	a2	b
a1	3	1	0	0	1	0	3
a2	4	3	-1	0	0	1	6
f3	1	2	0	1	0	0	4
W	**0**	**0**	**0**	**0**	**1**	**1**	**0**

Fase 1, montagem da linha W: as artificiais a1 e a2 ainda têm 1 na linha W.

Base	x1	x2	e2	f3	a1	a2	b
a1	**3**	1	0	0	1	0	3
a2	4	3	-1	0	0	1	6
f3	1	2	0	1	0	0	4
W	**-7**	**-4**	**1**	**0**	**0**	**0**	**-9**

Depois de subtrair da linha W as linhas de a1 e a2: entra x1 (−7), sai a1 (menor razão 1).

Depois de mais uma iteração W chega a 0 e a Fase 1 termina. Na Fase 2 as colunas `a1` e `a2` somem e entra a função objetivo original:

Base	x1	x2	e2	f3	b
x1	1	0	1/5	0	3/5
x2	0	1	-3/5	0	6/5
f3	0	0	1	1	1
Z	**4**	**1**	**0**	**0**	**0**

Fase 2, montagem da linha Z (ainda antes de zerar os coeficientes de x1 e x2).

Resultado final do programa: `x1 = 2/5`, `x2 = 9/5`, **Z mínimo = 17/5 (3,4)**.

6. `algoritmo_grafico.py`: Método Gráfico

Só funciona com **2 variáveis** (se tiver mais, avisa para usar o Simplex). A classe `MetodoGrafico` segue os passos do método gráfico da aula, e as funções soltas do arquivo fazem a geometria.

Função	Passo do método	O que faz
`resolver`	Roteiro geral	Junta as restrições do problema com `x1 ≥ 0` e `x2 ≥ 0` e executa os passos abaixo em ordem.
`_etapa_retas`	1º Tabela das retas	Troca cada sinal por `=` para obter a **reta**, calcula onde ela corta os eixos (faz x1 = 0 e depois x2 = 0) e faz o **teste da origem**.
`_corte_no_eixo`	1º	Calcula `b ÷ coeficiente`; se o coeficiente for 0, a reta "não corta" aquele eixo.
`_pontos_da_reta`	1º	Os dois pontos usados para desenhar a reta. Trata reta vertical, horizontal e reta que passa pela origem.
`_teste_da_origem`	1º	Substitui (0, 0) na restrição: se der verdadeiro, a região fica do lado da origem; se der falso, do lado oposto.
`_etapa_montagem`	2º Montar o gráfico	Explica como desenhar: 1º quadrante, ligar os pontos, escolher o lado de cada reta e pintar a **região viável**.
`restricoes_nao_negatividade`	2º	Cria `x1 ≥ 0` e `x2 ≥ 0` como retas comuns (são os eixos).
`intersecao`	3º Vértices	Encontra o ponto onde duas retas se cruzam (regra de Cramer). Retas paralelas → `None`.
`calcular_vertices`	3º Vértices	Cruza **todas as retas duas a duas** e fica só com os pontos que atendem **todas** as restrições. Esses são os vértices da região viável. Nenhum vértice → **inviável**.
`ordenar_vertices`	Desenho	Ordena os vértices em volta do centro para formar o contorno do polígono pintado.
`direcoes_ilimitadas` `_direcao_respeita`	Caso especial	Procuram direções em que a região viável vai até o infinito (região aberta).
`_objetivo_cresce_sem_limite`	Caso especial	Se Z melhora andando numa dessas direções, o problema é **ilimitado**.
`_etapa_vertices`	4º Avaliar Z	Tabela com cada vértice, quais retas se cruzam nele e o valor de Z. Destaca o melhor.
`_escolher_otimo`	4º Conclusão	Pega o **maior** Z (max) ou o **menor** (min). Se dois vértices empatam, avisa que há **soluções múltiplas** (todo o segmento entre eles é ótimo).

Exemplo: o mesmo `max z = 3x1 + 5x2` pelo gráfico

Reta	Equação da reta	Se x1 = 0	Se x2 = 0	Pontos	Teste (0, 0)
R1	x1 = 4	não corta	x1 = 4	(4, 0), reta vertical	0 ⩽ 4 (verdadeiro: lado da origem)
R2	2x2 = 12	x2 = 6	não corta	(0, 6), reta horizontal	0 ⩽ 12 (verdadeiro: lado da origem)
R3	3x1 + 2x2 = 18	x2 = 9	x1 = 6	(0, 9) e (6, 0)	0 ⩽ 18 (verdadeiro: lado da origem)

1º passo: tabela das retas.

Vértice	Retas que se cruzam	x1	x2	Z
P1	eixo x2 (x1 = 0) e eixo x1 (x2 = 0)	0	0	0
P2	R1 e eixo x1 (x2 = 0)	4	0	12
P3	R1 e R3	4	3	27
P4	R2 e R3	2	6	36
P5	R2 e eixo x2 (x1 = 0)	0	6	30

3º e 4º passos: vértices da região viável e Z em cada um; P4 é o melhor.

```
O maior valor de Z entre os vértices é 36, obtido em (2, 6).

Solução ótima: x1 = 2, x2 = 6, Z = 36
```

O resultado bate com o do Simplex (x1 = 2, x2 = 6, Z = 36), como deveria.

7. Onde cada caso especial é detectado

Situação	Simplex / Duas Fases	Gráfico
Ótimo	`coluna_que_entra` não acha negativo na linha Z	`_escolher_otimo` pega o melhor vértice
Ilimitado	`linha_que_sai` não acha razão válida (coluna sem positivos)	`_objetivo_cresce_sem_limite`
Inviável	Fim da Fase 1 com **W > 0** (`_executar_fase_1`)	`calcular_vertices` não encontra nenhum vértice
Soluções múltiplas	Não básica com 0 na linha Z (`nao_basicas_com_custo_zero`)	Dois ou mais vértices com o mesmo Z ótimo
Degeneração / ciclo	Base repetida → regra de Bland (`executar_iteracoes`)	Não se aplica
Restrição redundante	`_retirar_artificiais_da_base` remove a linha	A reta simplesmente não forma vértice

8. Resumo rápido dos métodos

- **Método Gráfico**: só 2 variáveis. Desenha as retas, acha a região viável e testa Z em cada vértice. O ótimo está sempre num vértice.
- **Simplex**: qualquer número de variáveis, todas as restrições `⩽`. Parte da origem (só folgas na base) e vai trocando uma variável por vez (entra o mais negativo de Z, sai a menor razão) até não sobrar negativo na linha Z.
- **Duas Fases**: Simplex para quando há `⩾` ou `=`. A Fase 1 serve só para achar um ponto de partida viável (zerar as artificiais); a Fase 2 é o Simplex normal a partir desse ponto.
- **Forma padrão** (`algoritmo_pl.py`): preparação comum ao Simplex e ao Duas Fases (min → max, b ≥ 0, folga/excesso/artificial).

Todos os quadros e tabelas deste documento foram gerados rodando o próprio código da pasta `simplexlab/algoritmos`.